

GUÍA DE INGENIERÍA Y CALIDAD



Guía de Convenciones y Estándares de Desarrollo

Proyecto: Suite Gamificada de Bienestar Digital (Android / Kotlin)

Estilo Visual: Kinetic Neo-Brutalist Dark & Bento Grid

Target: Adolescentes (13 a 16 años)

Stack: Kotlin 2.0+ / Android Studio / Jetpack Compose

1. Arquitectura y Estructura del Proyecto

Se adopta una arquitectura modular basada en **Clean Architecture + MVVM (Model-View-ViewModel)** y **UDF (Unidirectional Data Flow)**, recomendada por el equipo de Android:

PLAINTEXT

```
com.bienestardigital.app/  
├── core/  
│   ├── designsystem/ // Tokens de diseño (Colores, Tipografía, Formas)  
│   │   ├── Color.kt  
│   │   ├── Type.kt  
│   │   ├── Shape.kt  
│   │   └── Theme.kt  
│   ├── components/ // Componentes UI reutilizables estilo Neo-Brutalista  
│   │   ├── BentoCard.kt  
│   │   ├── BrutalistButton.kt  
│   │   ├── StatChip.kt  
│   │   └── DopamineProgressBar.kt  
│   └── common/ // Extensiones, utilitarios y dispatchers  
├── data/ // Repositorios, fuentes de datos locales (Room / DataStore)  
│   ├── local/  
│   └── repository/  
├── domain/ // Modelos de dominio y casos de uso (UseCases)  
│   ├── model/  
│   └── usecase/  
├── feature/ // Features / Módulos de pantalla  
├── onboarding/ // Registro de perfil inicial (Paso 1 y 2)  
├── hub/ // Menú principal Bento Grid  
├── decisionesconectadas/ // Minijuego 1: Narrativa Interactiva  
├── cazanotificaciones/ // Minijuego 2: Reacción y Alertas  
├── desmitificador/ // Minijuego 3: Swipe Cards Mito o Verdad  
└── organizador24h/ // Minijuego 4: Balance de Tiempo y Sueño
```

2. Convenciones Oficiales de Kotlin (Coding Conventions)

Todo el código debe cumplir estrictamente con la guía de estilo oficial de Kotlin y las recomendaciones de Android:

2.1. Nomenclatura (Naming Conventions)

- **Clases, Interfaces y Objetos:** PascalCase (ej. `BentoCardDefaults`, `DecisionesViewModel`, `NotificationItem`).
- **Funciones Composable:** PascalCase con sustantivos o adjetivos si emiten UI (ej. `HubScreen`, `BrutalistButton`). Si devuelven un valor o computan datos, usar camelCase.
- **Funciones y Métodos estándar:** camelCase con verbos de acción (ej. `onOptionSelected()`, `calculateDailyBalance()`, `evaluateSwipeDirection()`).
- **Variables y Propiedades:** camelCase (ej. `currentScore`, `streakCount`, `remainingSeconds`).
- **Constantes (const val o miembros inmutables de object / companion object):** SCREAMING_SNAKE_CASE (ej. `MAX_DAILY_HOURS`, `RECOMMENDED_SLEEP_HOURS`).
- **Backing Properties (Estado mutable interno vs immutable público):** Usar prefijo guion bajo para la variable privada mutable:

KOTLIN

```
private val _uiState = MutableStateFlow(HubUiState())  
val uiState: StateFlow<HubUiState> = _uiState.asStateFlow()
```

2.2. Buenas Prácticas y Calidad de Código

- **Inmutabilidad por defecto:** Priorizar `val` sobre `var`. Utilizar colecciones inmutables (`List`, `Set`, `Map`) en lugar de mutables (`MutableList`).
- **Manejo Seguro de Nulabilidad:** Evitar el operador de aserción no nula (`!!`). Favorecer el operador elvis (`? :`), llamadas seguras (`?.`) y asignaciones seguras con `let`.
- **Formato y Espaciado:**
 - Indentación estricta de 4 espacios (no usar tabs).
 - Longitud máxima recomendada de línea: 100 a 120 caracteres.
 - Encadenamiento de métodos: cada llamada en una línea separada precedida por un punto.

3. Tokens de Diseño y Recursos UI (DESIGN.md)

Para mantener la fidelidad del diseño **Kinetic Neo-Brutalist Dark**, no se deben escribir valores hexadecimales ni dimensiones sueltas ("hardcodeadas") en las vistas. Todos deben extraerse del sistema centralizado.

3.1. Paleta de Colores (Color.kt)

KOTLIN

```
package com.bienestardigital.app.core.designsystem

import androidx.compose.ui.graphics.Color

object AppColors {
    // Canvas & Superficies Base
    val CanvasBase = Color(0xFF0E0E12)
    val SurfaceElevated = Color(0xFF1B1B24)
    val SurfaceSunken = Color(0xFF14141B)
    val SurfaceHighlight = Color(0xFF262634)

    // Bordes Estructurales Neo-Brutalistas
    val StrokeStructural = Color(0xFF2E2E3E)
    val StrokeSubtle = Color(0xFF20202C)
    val StrokeUrgent = Color(0xFFFFF5733)

    // Acentos Cromáticos Eléctricos
    val ElectricLime = Color(0xFFCCFF00) // Acción principal / CTA
    val NeonCyan = Color(0xFF00F5D4) // Reflejos / Respuestas correctas
    val DigitalViolet = Color(0xFF9D4EDD) // Narrativa / Sueño / XP
    val AlertCoral = Color(0xFFFFF5733) // Trampas de retención / Alertas
    val GoldenStar = Color(0xFFFFFD166) // Rachas e insignias

    // Tipografía y Contraste
    val TextPrimary = Color(0xFFFF4F4F8)
    val TextSecondary = Color(0xFF9D9DAE)
    val TextOnLime = Color(0xFF0E0E12) // Contraste AAA sobre ElectricLime
}
```

3.2. Tipografía (Type.kt)

Space Grotesk: Reservada exclusivamente para Display, titulares (headline), contadores numéricos, temporizadores y badges.

Plus Jakarta Sans: Reservada para narrativa, cuerpo de notificaciones, descripciones y datos científicos.

KOTLIN

```

package com.bienestardigital.app.core.designsystem

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

val SpaceGrotesk = FontFamily(
    Font(R.font.space_grotesk_bold, FontWeight.Bold)
)
val PlusJakartaSans = FontFamily(
    Font(R.font.plus_jakarta_sans_medium, FontWeight.Medium),
    Font(R.font.plus_jakarta_sans_bold, FontWeight.Bold)
)

val AppTypography = Typography(
    displayLarge = TextStyle(
        fontFamily = SpaceGrotesk,
        fontWeight = FontWeight.Bold,
        fontSize = 36.sp,
        lineHeight = 40.sp,
        letterSpacing = (-0.03).sp
    ),
    headlineMedium = TextStyle(
        fontFamily = SpaceGrotesk,
        fontWeight = FontWeight.Bold,
        fontSize = 22.sp,
        lineHeight = 28.sp,
        letterSpacing = (-0.02).sp
    ),
    bodyLarge = TextStyle(
        fontFamily = PlusJakartaSans,
        fontWeight = FontWeight.Medium,
        fontSize = 16.sp,
        lineHeight = 24.sp
    ),
    labelSmall = TextStyle(
        fontFamily = SpaceGrotesk,
        fontWeight = FontWeight.Bold,
        fontSize = 10.sp,
        lineHeight = 14.sp,
        letterSpacing = 0.06.sp
    )
)

```

3.3. Elevación, Bordes y Profundidad Táctil

PROHIBIDO

Sombras difusas (blur suave o elevation estándar con gradiente borroso).

REGLA OBLIGATORIA

Usar sombras duras planas (drop shadow sin desenfoque): 0 . dp X, 4 . dp Y en color #000000 o #2E2E3E.

FEEDBACK DE PULSADO (MICRO-INTERACCIÓN TÁCTIL)

Todo botón interactivo al ser presionado debe desplazarse 4 . dp hacia abajo (`offset(y = 4 . dp)`) reduciendo la sombra a 0 . dp.

- **Bordes de Contenedores Bento:** 20 . dp de radio (`RoundedCornerShape(20 . dp)`) con borde sólido de 1 . 5 . dp a 2 . dp en `StrokeStructural`.
- **Pills y Botones:** Redondeo completo `RoundedCornerShape(9999 . dp)`.

4. Convenciones para Jetpack Compose y UI

- **State Hoisting:** Ningún composible de UI directa debe retener estado interno si este altera la lógica de negocio. Separar composables con estado (*Stateful*) de composables puramente visuales (*Stateless*):

KOTLIN (STATELESS COMPOSABLE)

```
// STATELESS COMPOSABLE (Reutilizable y testeable)
@Composable
fun DecisionCard(
    text: String,
    impactBadgeText: String,
    modifier: Modifier = Modifier,
    onClick: () -> Unit
) { // ... }
```

- **Parámetro Modifier estándar:** Todo Composable de vista debe recibir un `modifier: Modifier = Modifier` como primer parámetro opcional y aplicarlo a su contenedor raíz.
- **Optimización de Renderizado:** Usar `remember` y `derivedStateOf` para cálculos intensivos dentro de la fase de composición (como el cálculo del progreso de tiempo en el minijuego de 24 horas).

5. Gestión del Flujo de Trabajo en Git

Para asegurar que todo el equipo mantenga la estabilidad del repositorio:

Estrategia de Ramas (GitHub Flow / GitFlow simplificado):

- `main`: Código listo para producción/entrega final.
- `develop`: Rama de integración principal.
- `feature/<nombre-feature>`: Ramas de trabajo para cada funcionalidad o pantalla. Ejemplos:
 - `feature/hub-bento-grid`
 - `feature/game-decisiones-conectadas`
 - `feature/game-caza-notificaciones`
 - `feature/game-desmitificador`
 - `feature/game-organizador-24hs`
 - `feature/onboarding-profile`

Convención de Commits (Conventional Commits):

- `feat`: Nueva característica (ej. `feat(hub): add streak counter and daily hero challenge`).
- `fix`: Corrección de error (ej. `fix(organizador24h): prevent sleep hours from being reduced below zero`).
- `style`: Ajustes visuales, colores, padding o tipografía sin impacto en la lógica.
- `refactor`: Mejora interna de código sin alterar comportamiento visual ni lógica de negocio.
- `docs`: Cambios en la documentación o especificaciones.

Pull Requests (PR):

- Ningún commit directo a `main` o `develop`.
- Todo PR debe ser revisado y aprobado por al menos 1 compañero de equipo antes de integrarse.
- Antes de solicitar revisión, ejecutar el linter local (`ktlint` o inspección de código de Android Studio).